

RallyPoint

Milestone 1 演示视频脚本 · Demo Video Script (Front-End Component)

总时长约 6:25

英文口播约 965 词

覆盖: Demonstration /20 · Code Walkthrough /15 · Discussion of Issues & Resolutions /15

本里程碑 = ACIS 498 Milestone 1 前端组件 (React/TypeScript) ; 后端 Flask + DB 属于 Milestone 2

全程真实页面与真实代码, 无夸大

开场 OPENING | 0:00 – 0:46

【画面】 深色模式打开 RallyPoint, 停在登录页。

Hi, I'm Phoebe — this is RallyPoint, my ACIS 498 capstone: an AI partner-matcher for everyday tennis and pickleball players in a city. Regular players struggle to find someone who fits — close on skill, free at the same hours, near the same public court — so good courts sit half-empty.

【画面】 鼠标扫过顶部 Find Partners / My Games / Courts。

My matching is a transparent, explainable score over real signals — skill, distance, schedule overlap, shared courts — plus a genuine AI signal: a "playing style" match from bio embeddings. The only optional piece, an LLM that rewrites wording, is off here. This milestone is the React front end; I'll show the product, the code, and the issues I solved.

(约 110 词)

演示 DEMONSTRATION | 0:46 – 2:52 | /20

【画面】 登录 (alex@rally.app / rally1234) 。

Let's log in — the live React app talking to the API over a typed, cookie-authenticated client.

【画面】 Find Partners: 停在排好序的候选卡片, 鼠标依次指 tier、% 分数、各 reason chip。

This is Find Partners. Each match shows a tier — great, good, or worth a try — a percent score, and reason chips, every chip a real signal the score is built from: "same level," "a mile away," "three shared time slots," even "similar playing style," which comes from comparing the two players' bio embeddings. Best matches on top.

【画面】 拖 NTRP 滑块、切换 sport, 列表实时更新。

Filtering is live — I drag the skill range or switch sport, and the list re-ranks itself, no submit button.

【画面】 在一张卡片上发起约球 → 弹出邀请: 选时间 + 选球场 (CourtPicker) 。

To play, I send a request as a two-phase invite: I propose a time and pick a real court, and the other player confirms or counters — a request is never faked into a booked game.

【画面】 切到 Profile: preferred-times 网格 (AvailabilityGrid)、neighborhood、NTRP/DUPR、saved players。

On my profile, this weekly preferred-times grid sets when I play — the engine scores how much two players' grids overlap. Plus my Chicago neighborhood, my real NTRP and DUPR ratings, and saved players that live on my account, not just this browser.

【画面】 切到 My Games: upcoming / requests (带 badge) / past, 在一条 request 上 Accept。

My Games is the timeline — upcoming, incoming requests badged from any page, and past. I can accept, decline, or propose a new time.

【画面】 Courts: 真实芝加哥球场 + 地图; 点开一个进 Court Detail (/courts/:slug), 展示 "N here now" 签到 + open games 的 Join / waitlist。

Courts are real Chicago public courts with real distance and a live map. Open one, and the court page shows who's here now from check-ins, plus open games you can join or waitlist — real coordination, no fake data.

【画面】 切到 Admin: 统计卡片 → Users 标签按 status / sport 筛选 → 打开一个普通用户的 Edit (评分/地址/删除)。最后点 dark-mode。

And because I'm an admin, there's a dashboard: live stats, a user list I filter by status and sport, and a full edit — ratings, address, even delete — for trust and safety. One last thing: dark mode, clean across the whole app.

(约 320 词)

代码讲解 CODE WALKTHROUGH | 2:52 – 4:30 | /15

【画面】 打开 frontend/src/api/client.ts, 高亮 resolveApiBase()、credentials:"include"、X-CSRF-Token、401 的 auth:expired。

The front-end's foundation is one typed API layer. Every call goes through a single fetch wrapper; the base URL comes from a Vite env var that Vercel injects in production, with localhost only as a dev fallback. Auth is an httpOnly cookie plus an X-CSRF-Token double-submit, so JavaScript never holds the token, and a 401 fires a global auth:expired event.

【画面】 打开 api/players.ts + hooks/usePlayers.ts, 高亮 queryKey:["players", filters]、matchTier/matchScore/matchReasons。

This is the call that fetches the AI matches. A TanStack Query hook keys on the filters, so changing them refetches and caches a result per combination — that drives skeletons versus cards, no manual loading flags.

【画面】 打开 contexts/AuthContext.tsx + components/ProtectedRoute.tsx。

AuthContext is the single source of truth for the user: it validates the cookie on mount and clears on auth:expired and cross-tab logout. ProtectedRoute gates every authenticated page and adds an email-verification gate.

【画面】 打开 pages/FindPartnerPage.tsx, 高亮 300ms debounced useEffect 与 match-tier/.match-chips 渲染。

Find Partners renders the backend's tier, score, and reason chips verbatim — never recomputed here — behind a 300-millisecond debounce, so dragging the slider fires one query, not one per tick.

【画面】快速划过 components/Modal.tsx、AvailabilityGrid.tsx、CourtPicker.tsx，以及 favorite / saved-player / check-in 的 optimistic 更新。

The UI is built from shared primitives — one Modal, the availability grid, the court picker — and optimistic mutations for favorites, saved players, and check-ins that roll back on error. Types flow end to end, from the API client to the JSX. One honest bridge: the scoring engine itself, including the embeddings, runs in the Flask backend — that's Milestone 2. Here the front end consumes it.

(约 235 词)

问题与解决 ISSUES & RESOLUTIONS | 4:30 – 6:00 | /15

【画面】打开 client.ts 高亮 resolveApiBase(): import.meta.env.DEV 门控 + 生产抛错。

Three real front-end issues. First, a deployment trap: my API client used to fall back to localhost whenever VITE_API_URL was missing — fine locally, but in production that asked each user's own browser to call their localhost, a silent failure. I fixed it with resolveApiBase: the fallback exists only in Vite dev, and production throws loudly if the variable is missing.

【画面】高亮 sample fallback 的 DEV || VITE_DEMO_FALLBACK 门控；切两张状态图 ("unavailable, Try Again" / "no matches")。

Second, keeping the UI honest. A sample-player fallback made the page look alive when the backend was down — but in production a user could mistake fake players for real ones. So samples are gated to dev only; a production failure shows a clear "unavailable, try again," and a true empty result shows "no matches" — never fabricated users.

【画面】打开 hooks/useSavedPlayers.ts 高亮 onMutate/onError；现场点一次 save 即时翻转。

Third, real preferences with instant feedback. Bookmarks used to live only in localStorage — device-specific and temporary. I moved them to a server-backed API with an optimistic mutation: the bookmark flips instantly and rolls back on error, so saved partners follow your account. That same pattern drives court favorites and check-ins. The backend setup issues — a circular import, a dependency build — were real too, but those belong to Milestone 2.

(约 235 词)

结尾 CLOSING | 6:00 – 6:25

【画面】回到 Find Partners，停在带 reason chips 与 % 分数的卡片上。

So what creates the most value? A transparent, explainable score — with a genuine embedding signal for playing style, auditable and no black box — wrapped in a React front end that turns it into a real loop: match, invite, check in, play. Next is Milestone 2: the Flask engine in depth, and wiring ai_match_logs — designed but not yet logging — into a learning-to-rank loop. Thanks for watching.

(约 65 词)

英文口播合计约 965 词 (110 + 320 + 235 + 235 + 65)；按 ~150 词/分钟约 6:25，留足余量在 7:00 内。若现场偏长，优先压缩 Demo 的 Admin/My Games 两段或 Issues 的根因叙述。

录制小贴士

- 开场定调讲清楚："matching 是可解释的分数（技能/距离/时段重叠/球场）+ 真 embedding 的'打法相似度'信号；唯一可选的在线 LLM 只重写措辞、本演示关闭"，"本里程碑是 React 前端"。
- Find Partners：鼠标依次停在 tier、% 分数、每个 reason chip 上。"similar playing style" chip 来自 bio embedding，只有种子数据算过 embedding（部署时设了 OPENAI_API_KEY）才出现——录前确认线上 demo 有没有这个 chip：有就指着讲，没有就只讲架构、别声称它在画面上。availability 说成真实的每周 preferred-times 网格重叠（已不是关键词匹配）。
- 两阶段邀请：强调"propose 时间 + 选球场、对方确认/还价"，不是直接生成已确认的比赛——诚实点正好加分。
- Courts / Court Detail：只口播已核实字段（distance、map、here-now 签到、open games）；约局/签到是"真实活动、不伪造预订"。
- Admin 段为可选控时：若整体偏长，这段可压成一句带过；删除演示请删一个普通测试用户，不要删你自己的 admin（端点会拒绝删自己）。
- Code Walkthrough：按 client.ts → players.ts/usePlayers.ts → AuthContext/ProtectedRoute → FindPartnerPage → Modal/AvailabilityGrid/CourtPicker 排好标签页；认证务必说 httpOnly cookie + CSRF、生产 API 地址由 Vercel env var 指向 Render。明说一次"scoring engine 含 embeddings 在 Flask 后端、属于 Milestone 2"，不展开后端。
- Issues：两张状态图（"unavailable, Try Again" / 空状态 "no matches"）事先截好，不现场断网。
- 全程只展示真实页面与真实代码；ai_match_logs 仅作"下一步"且明说尚未 logging；不声称在线 LLM 在跑；不声称前端持有 token（httpOnly cookie + CSRF）。